Ders 21 Çalışma Özeti

Ders 21 Çalışma Özeti

Genel Konular

- Dersin tarihi 6 Aralık 2021 Pazartesi. Ders çevrim içi (Zoom) yapıldı.
  - Slaytlar Silberschatz, Galvin ve Gagne, *Operating System Concepts*, 9. baskı. Bölüm 5'in sonu ve Bölüm 6'nın başı.
  - Asıl anlatım yaklaşık 00:07'de başlar. Ders 00:58-01:12 arasında ara verir.
- Klasik senkronizasyon problemleri (Bölüm 5)
  - Sınırlı tampon (bounded buffer) problemi: üç semaforla çözüm.
  - Okuyucu-yazıcı (readers-writers) problemi: birinci ve ikinci varyasyon, okuyucu-yazıcı kilidi.
  - Yemek yiyen filozoflar (dining philosophers) problemi: basit semafor çözümü, kilitlenme ve çözüm yolları.
- Monitör (monitor)
  - Yapısı, koşul değişkenleri (condition variable), `wait` ve `signal`.
  - "Signal and wait" ve "signal and continue" yaklaşımları.
  - Filozoflar probleminin monitörle kilitlenmesiz çözümü.
  - Monitörde bekleyen süreçlerin sırası: FCFS veya öncelikli bekleme (`x.wait(c)`).
- İşletim sistemlerinde senkronizasyon örnekleri
  - Solaris uyarlanabilir mutex (adaptive mutex), Windows, Linux, Pthreads.
- İşlemci zamanlama (CPU scheduling, Bölüm 6)
  - CPU ve G/Ç patlamaları (CPU burst, I/O burst), CPU'ya bağlı (CPU-bound) ve G/Ç'ye bağlı (I/O-bound) süreçler.
  - Kısa vadeli zamanlayıcı (short-term scheduler), zamanlama kararının verildiği dört durum, kesintili (preemptive) ve kesintisiz (non-preemptive) zamanlama.
  - Dağıtıcı (dispatcher) ve dağıtım gecikmesi (dispatch latency).
  - Zamanlama ölçütleri: CPU kullanımı, verim (throughput), tamamlanma süresi (turnaround time), bekleme süresi (waiting time), yanıt süresi (response time).
  - Algoritmalar: FCFS, SJF, sonraki CPU patlamasının üstel ortalama ile tahmini, SRTF, öncelikli (priority) zamanlama.
- Round robin ve çok seviyeli kuyruklar sonraki derse bırakıldı.

Hocanın Özellikle Vurguladığı Kısımlar

- Senkronizasyon mekanizmasını sağlamak işletim sisteminin, onu doğru kullanmak uygulama geliştiricinin sorumluluğudur.
  - Monitör de soyut, kolay kullanımlı bir yapıdır. Ama yanlış kullanılırsa yine kilitlenme ve açlık oluşur.
  - Semaforda `wait` ve `signal`'ın yer değiştirmesi, ikisinden birinin unutulması ya da iki kez çağrılması tipik hatalardır. Aynı hatalar monitörde koşul değişkenleriyle de yapılabilir.
- Filozoflar çözümündeki kilitlenme her zaman oluşmaz.
  - Herkes aynı anda sol çubuğu alırsa oluşur. Olma ihtimalinin olması yeterince kötüdür, çünkü böyle hataları bulmak zordur.
- Ödev niteliğinde düşünme sorusu: Monitörle yazılan kilitlenmesiz filozoflar çözümünde açlık neden oluşur ve nasıl giderilir? Hoca bunu ikinci vizeye kadar düşünmenizi istedi.
- FCFS'de uzun bir iş öne gelirse kısa işler uzun süre bekler.
  - Ortalama bekleme süresi çok artar. Bu duruma konvoy etkisi (convoy effect) denir.

- CPU patlamalarının çoğu kısadır.
  - Hocanın gösterdiği histogramda patlamaların büyük kısmı 8 ms'nin altında. Zamanlama algoritmaları bu dağılım göz önüne alınarak değerlendirilir.
- Patlama süreleri önceden bilinmez, tahmin edilir.
  - Örneklerde verilen patlama süreleri tahmini değerlerdir.
- Öncelikli zamanlamanın asıl sorunu düşük öncelikli sürecin açlığıdır.
  - Çözüm, bekledikçe önceliği artırmaktır (yaşlandırma, aging).
- Öncelik değerinin yönü sisteme göre değişir.
  - Bazı sistemlerde küçük sayı yüksek öncelik, bazılarında büyük sayı yüksek önceliktir.

Kısa Tekrar Notları

- Sınırlı tampon: `mutex = 1`, `full = 0`, `empty = n`.
  - Üretici: `wait(empty)`, `wait(mutex)`, ekle, `signal(mutex)`, `signal(full)`.
  - Tüketici: `wait(full)`, `wait(mutex)`, çıkar, `signal(mutex)`, `signal(empty)`.
- Birinci okuyucu-yazıcı problemi: Okuyucular önceliklidir. Yazıcılar açlığa düşebilir.
- İkinci okuyucu-yazıcı problemi: Yazıcılar önceliklidir. Okuyucular açlığa düşebilir.
- Filozoflar kilitlenmesinden kaçınma yolları:
  - Masaya en fazla 4 filozof oturt.
  - Filozof iki çubuğu yalnız ikisi de boşsa (kritik bölge içinde) alsın.
  - Asimetrik çözüm: tek numaralı filozof önce sol, sonra sağ çubuğu; çift numaralı önce sağ, sonra sol çubuğu alır.
- Monitör: Herhangi bir anda içinde yalnız bir süreç aktif olabilir.
  - `x.signal()`, `x` üzerinde bekleyen süreç yoksa hiçbir etki yapmaz. Semafordaki `signal` ise değeri her zaman artırır.
- Zamanlama kararı dört durumda verilir:
  - (1) çalışan → bekleyen, (2) çalışan → hazır, (3) bekleyen → hazır, (4) sonlanma.
  - Yalnız (1) ve (4) kullanılıyorsa zamanlama kesintisizdir.
- Tamamlanma süresi = bitiş zamanı − geliş zamanı.
- Bekleme süresi = tamamlanma süresi − CPU patlama süresi. Hazır kuyruğunda geçen toplam süre.
- Yanıt süresi = gelişten ilk yanıta kadar geçen süre. Etkileşimli sistemlerde önemlidir.
- SJF, ortalama bekleme süresini en aza indirir. SRTF, SJF'nin kesintili hâlidir.
- Üstel ortalama: $\tau_{n+1} = \alpha\, t_n + (1-\alpha)\, \tau_n$.

Detaylı Açıklamalar

Okuyucu-yazıcı çözümü (birinci varyasyon)

```
/* ortak: semaphore rw_mutex = 1, mutex = 1; int read_count = 0; */

/* yazıcı */
wait(rw_mutex);
/* yazma */
signal(rw_mutex);

/* okuyucu */
wait(mutex);
read_count++;
if (read_count == 1) wait(rw_mutex);   /* ilk okuyucu yazıcıları kilitler */
signal(mutex);
/* okuma */
wait(mutex);
read_count--;
if (read_count == 0) signal(rw_mutex); /* son okuyucu kilidi bırakır */
signal(mutex);
```

`mutex` yalnız `read_count` değişkenini korur. Okuyucu okumaya başlamadan `mutex`'i bırakır, böylece başka okuyucular da girebilir. Okuyucu olduğu sürece yazıcı giremez. Bu çözüm kilitlenme yaratmaz, ama yazıcıların açlığını önlemez.

Filozoflar: basit çözüm ve kilitlenme

```
/* semaphore chopstick[5], hepsi 1 */
while (true) {
    wait(chopstick[i]);
    wait(chopstick[(i + 1) % 5]);
    /* ye */
    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);
    /* düşün */
}
```

Beş filozof aynı anda ilk `wait`'i yapıp kesilirse herkes bir çubuk tutar ve ikincisini bekler. Kilitlenmenin dört şartı (karşılıklı dışlama, tut ve bekle, kesinti yok, döngüsel bekleme) sağlanmış olur.

Monitörle filozoflar

- Durumlar: `THINKING`, `HUNGRY`, `EATING`. Her filozof için bir koşul değişkeni `self[i]` vardır.
- `pickup(i)`: Durumu `HUNGRY` yap, `test(i)` çağır. Yemeye başlayamıyorsa `self[i].wait()`.
- `test(i)`: Sol ve sağ komşu yemiyorsa ve i açsa durumu `EATING` yap ve `self[i].signal()`.
- `putdown(i)`: Durumu `THINKING` yap, sonra iki komşu için `test` çağır.
- Bu çözüm kilitlenmesizdir ama açlık olabilir: iki komşu sırayla yerse aradaki filozof hiç yiyemeyebilir.

Zamanlama örnekleri

FCFS, geliş sırası P_1, P_2, P_3, patlamalar 24, 3, 3:

Süreç	Bekleme
P_1	0
P_2	24
P_3	27

Ortalama $(0 + 24 + 27)/3 = 17$. Sıra P_2, P_3, P_1 olsaydı bekleme 6, 0, 3 ve ortalama 3 olurdu.

SJF, patlamalar $P_1 = 6$, $P_2 = 8$, $P_3 = 7$, $P_4 = 3$. Sıra P_4, P_1, P_3, P_2:

Süreç	Bekleme
P_1	3
P_2	16
P_3	9
P_4	0

Ortalama $28/4 = 7$. Aynı süreçler FCFS ile çalışsa bekleme 0, 6, 14, 21 olur. Ortalama $41/4 = 10{,}25$.

Üstel ortalama ile tahmin, $\alpha = 1/2$, $\tau_0 = 10$. Gerçek patlamalar 6, 4, 6, 4, 13, 13, 13:

t_n	6	4	6	4	13	13	13
τ_{n+1}	8	6	6	5	9	11	12

Tahmin, gerçek değerlere yavaş yavaş yaklaşır.

SRTF (slayt 6.17): P_1 (0, 8), P_2 (1, 4), P_3 (2, 9), P_4 (3, 5), parantezde (geliş, patlama). Gantt: P_1 0-1, P_2 1-5, P_4 5-10, P_1 10-17, P_3 17-26. Ortalama bekleme $[(10-1)+0+(17-2)+(5-3)]/4 = 26/4 = 6{,}5$ ms.

Öncelikli zamanlama: patlamalar P_1–P_5 için 10, 1, 2, 1, 5. Küçük sayı yüksek öncelik. Sıra P_2, P_5, P_1, P_3, P_4. Bekleme 6, 0, 16, 18, 1. Ortalama $41/5 = 8{,}2$.

Yaşlandırma örneği

Öncelik 0-127 arası olsun (0 en yüksek). Bekleyen sürecin önceliği her 15 dakikada bir artırılsın. 127'den 0'a çıkması 127×15 dakika, yani yaklaşık 32 saat sürer. Hoca bir şehir efsanesini de aktardı: MIT'de 1973'te kapatılan IBM 7094'te, 1967'den beri çalışmayı bekleyen düşük öncelikli bir süreç bulunmuş.

Ders sonrası notlar

- Hoca asimetrik çözümü "tek numaralılar önce sol sonra sağ çubuğu alır, çift numaralılar daha sonra dener" diye anlattı. Slayttaki (5.41) çözüm farklıdır: çift numaralılar sırayla beklemez, çubukları ters sırada (önce sağ, sonra sol) alır. Bu sıralama döngüsel beklemeyi kırar. Hocanın dile getirdiği "çift numaralılar aç kalabilir" kaygısı slayttaki çözüm için geçerli değildir.
- Zamanlama kararının dört durumunu hoca ters sırayla saydı ve sonra "yalnız 1 ve 4" dedi. Kitaptaki numaralamada 1 çalışan → bekleyen, 4 sonlanmadır. Kesintisiz zamanlama yalnız bu iki durumda karar verir.
- SJF örneğinde hoca "ilk sırada çalıştırılan P3" dedi. İlk çalışan süreç P_4'tür (patlaması 3).
- Hoca SJF örneğinin FCFS karşılığını "on küsur" diye bıraktı. Tam değer $41/4 = 10{,}25$'tir.
- Hoca SRTF örneğinin ortalama bekleme süresini söylemedi. Slayttaki değer 6,5 ms'dir.
- Hoca Windows için "çok işlemcili sistemlerde doğrudan spinlock" dedi. Kitaba göre Windows çekirdeği tek işlemcide kesmeleri maskeler, çok işlemcide spinlock kullanır. Kullanıcı düzeyinde ise dispatcher nesneleri (mutex, semafor, olay) sağlar.
- Altyazıdaki "pound profit" bounded buffer, "reader-siters" readers-writers, "DynicG0" dining philosophers, "trueput" throughput, "konsümer" consumer demektir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.